

Gauss–Newton Temporal Difference Learning With Nonlinear Function Approximation

Zhifa Ke, Junyu Zhang[✉], and Zaiwen Wen[✉], *Member, IEEE*

Abstract—In this article, a Gauss–Newton temporal difference (GNTD) learning method is proposed to solve the Q -learning problem with nonlinear function approximations. In each iteration, our method takes one Gauss–Newton (GN) step to optimize a variant of mean-squared Bellman error (MSBE), where target networks are adopted to avoid double sampling. Inexact GN steps are analyzed so that one can safely and efficiently compute the GN updates by cheap matrix iterations. Under mild conditions, nonasymptotic finite-sample convergence to the globally optimal Q function is derived for various nonlinear function approximations. In particular, for neural network parameterization with ReLU activation, GNTD achieves an improved sample complexity of $\tilde{\mathcal{O}}(\varepsilon^{-1})$, as opposed to the $\mathcal{O}(\varepsilon^{-2})$ sample complexity of the existing neural temporal difference (TD) methods. An $\tilde{\mathcal{O}}(\varepsilon^{-1.5})$ sample complexity of GNTD is also established for general smooth function approximations. We validate our method via extensive experiments on several reinforcement learning (RL) benchmarks, where GNTD exhibits both higher rewards and faster convergence than TD-type methods.

Index Terms—Bellman equation, Gauss–Newton (GN), reinforcement learning (RL), temporal difference (TD) learning.

I. INTRODUCTION

EVALUATING the Q function for a certain policy π , and learning the optimal Q function, are important targets in reinforcement learning (RL). During the past decades, under the neural networks function approximation, deep Q -learning algorithms [1], [2] have achieved impressive empirical success in large-scale practical problems. They also serve as a fundamental building block for popular RL methods like the trust region policy optimization [3] and the actor–critic algorithms [4], [5], [6], where a proper parameterization of the Q function can be crucial to the scalability of the algorithm. Common examples include linear [7], [8], [9] and neural network [10], [11], [12] function approximations. In this work,

we mainly study the Gauss–Newton (GN)-type semi-gradient methods for finding the evaluating Q functions for a fixed policy. The proposed method can be easily extended to optimal Q functions.

Let Q^π be the unknown state action value function to be estimated under policy π and let T^π be the corresponding Bellman operator. Then, the standard formulation for policy evaluation with function approximation is to minimize the mean-squared Bellman error (MSBE)

$$\min_{\theta} \mathcal{L}_{\mu}(\theta) = \|Q(\theta) - T^\pi Q(\theta)\|_{\mu}^2 = \mathbb{E}_{(s,a) \sim \mu} [(Q(s, a; \theta) - T^\pi Q(s, a; \theta))^2] \quad (1)$$

where μ is the *stationary distribution* of the state-action pairs under the policy π . A direct optimization of MSBE can be very hard, as the double sampling issue denies an unbiased estimator of $\nabla \mathcal{L}_{\mu}(\theta)$. A practical technology to address the double sampling issue is to twist the loss function by introducing a target parameter θ_{targ} as follows:

$$\mathcal{L}_{\mu}(\theta; \theta_{\text{targ}}) = \|Q(\theta) - T^\pi Q(\theta_{\text{targ}})\|_{\mu}^2. \quad (2)$$

Based on this technique, one popular class of methods for solving (1) are temporal difference (TD) [13] learning algorithm as well as its variants [14], [15], [16], [17]. Given the surrogate loss $\mathcal{L}_{\mu}(\theta; \theta_{\text{targ}})$ and the target parameter $\theta_{\text{targ}}^k = \theta^k$, the vanilla TD learning method updates by

$$\theta^{k+1} = \theta^k - \eta g^k$$

where g^k is an unbiased estimator of $\nabla_{\theta} \mathcal{L}_{\mu}(\theta^k; \theta_{\text{targ}}^k)$. Since $\mathbb{E}[g^k]$ only contains a part of $\nabla_{\theta} \mathcal{L}_{\mu}(\theta^k)$, TD is also termed as the semi-gradient method. Though TD-type algorithms have been empirically successful in many numerical applications [1], [5], [6], [18], the semi-gradient nature of TD-type methods has mostly limited their theoretical convergence guarantees to linear function approximations [15], [16]. Let us denote an ε -optimal solution to be a point $\bar{\theta}$ such that $\mathbb{E}[\|Q(\bar{\theta}) - Q^\pi\|_{\mu}^2] \leq \varepsilon$ or $\|Q(\bar{\theta}) - Q^\pi\|_{\mu} \leq \varepsilon$ with high probability. That is, when a nonsquared error measure is adopted, to be consistent, we say $\bar{\theta}$ is ε -optimal if $\mathbb{E}[\|Q(\bar{\theta}) - Q^\pi\|_{\mu}] \leq \sqrt{\varepsilon}$ or $\|Q(\bar{\theta}) - Q^\pi\|_{\mu} \leq \sqrt{\varepsilon}$ with high probability. Then, recent results on neural TD [10], [11] typically require a suboptimal $\mathcal{O}(\varepsilon^{-2})$ sample complexity to find some ε -optimal solution. For general smooth function approximation, TD-type methods have divergence issues both in theory and practice [19], [20], [21], [22]. Some variants, such as GTD2 and TDC [20], [23], [24], consider the projected mean-squared Bellman error (MSPBE) in the parameter space and establish asymptotic or finite-time analyses. However,

Received 21 February 2025; revised 1 September 2025; accepted 26 January 2026. Date of publication 25 February 2026; date of current version 2 September 2026. This work was supported in part by Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) under Grant A-0009530-05-00, in part by the National Key Research and Development Program of China under Grant 2024YFA1012900, and in part by the National Natural Science Foundation of China under Grant 12331010 and Grant 12288101. (Corresponding author: Junyu Zhang.)

Zhifa Ke is with Ubiquant Investment Corporation Ltd., Beijing 100084, China (e-mail: kezhifa@stu.pku.edu.cn).

Junyu Zhang is with the Department of Industrial Systems Engineering and Management, National University of Singapore, Singapore 117576 (e-mail: junyuz@nus.edu.sg).

Zaiwen Wen is with Beijing International Center for Mathematical Research, Peking University, Beijing 100871, China (e-mail: wenzw@pku.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2026.3661291>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2026.3661291

these algorithms can only prove that the parameters converge to a stationary point.

Another popular approach for TD-learning is the fit Q -iteration (FQI) method [25], [26], [27], which repeatedly solves a nonlinear least square subproblem to fit the Q -iteration

$$\theta^{k+1} \approx \operatorname{argmin}_{\theta} \mathcal{L}_{\mu}(\theta; \theta^k)$$

by sufficiently many stochastic gradient steps. Under expressive enough function approximation class such as over-parameterized neural networks, the FQI can largely approximate the contractive Q -iteration $Q(\theta^{k+1}) \approx T^{\pi}Q(\theta^k)$ and hence retain desirable convergence properties. However, the need to solve the nonlinear least square subproblems may lead to an increased demand for RL state-action-transition samples and the per iteration computation time, which makes FQI less competitive against TD in practice.

Interestingly, it can be observed that in the update of the TD learning method, the semi-gradient g^k coincides with the first-step gradient of the FQI's nonlinear least square subproblem. That is, $g^k = \nabla \mathcal{L}_{\mu}(\theta^k; \theta^k)$, and each iteration of the TD method can be viewed as an inexact FQI that takes only one stochastic gradient descent step to optimize the nonlinear least square subproblem. The inexactness here can be an interpretation of the divergence behavior of TD learning for general function approximations. This motivates us to design the GNTD learning algorithm, which takes only one GN step to optimize FQI's subproblem. The proposed method lies exactly in between FQI and TD, which requires fewer computations than FQI, while obtaining better theoretical convergence than TD.

A. Contributions

We provide a complete finite-time convergence analysis of GNTD with neural network and general smooth function approximations under the i.i.d. sampling setting. Leveraging the powerful theoretical properties of GNTD in nonlinear function approximation, we further explore its practical performance in real-world problems. Specifically, for neural networks, we design a new algorithm framework based on the Kronecker-factored approximate curvature (K-FAC) method [28]. This algorithm demonstrates exceptional numerical performance across both continuous and discrete tasks, as well as offline and online settings. We summarize our main contribution below.

- 1) We propose the GNTD learning algorithm to solve the Q learning problem with function approximation. We derive the convergence and sample complexity of our method under the i.i.d. sampling setting. Compared with the existing results of TD methods, GNTD achieves better sample complexities under nonlinear function approximations. For neural network approximation, GNTD achieves an improved $\tilde{\mathcal{O}}(\varepsilon^{-1})$ sample complexity as opposed the $\mathcal{O}(\varepsilon^{-2})$ sample complexity of the existing neural TD methods. Furthermore, as a by-product of the algorithm analysis, we also establish an

$\tilde{\mathcal{O}}(\varepsilon^{-1.5})$ sample complexity of GNTD to find the global optima for smooth function approximation.

- 2) We design an alternative implementation of the GN step based on the Levenberg–Marquardt method and K-FAC [28] approximation method, named GNTD-KFAC, which proves to be highly effective in practice. We conduct extensive experiments to numerically validate the efficiency of GNTD. For both online and offline RL tasks, our method outperforms TD and its variants, including the deep Q -network (DQN) method.

B. Related Work

TD learning was first proposed for policy evaluation and Q -learning [13], and it later developed numerous variants. TD learning with linear approximation has been extensively studied, including gradient TD (GTD) [15], [16], accelerated GTD [29], [30], and least-squares TD (LSTD) [14], [17], [31], [32]. Subsequently, GTD [20] was extended to the case of nonlinear function approximation. Since deep learning has achieved great success in large-scale problems, several variants based on neural network approximations have also been proposed, including DQN [1], preconditioned Q networks (PreQN) [21], and TD3 [6].

For the sample complexities of the nonlinear TD method and its variants, current results mainly focus on neural network function approximation and general smooth function approximations. For neural TD methods, the key observation is that wide over-parameterized neural networks are approximately linear under the neural tangent kernel (NTK) regime [33], [34], [35]. However, most existing neural TD methods only achieve the suboptimal $\mathcal{O}(\varepsilon^{-2})$ sample complexities [10], [11], [22]. A recent improvement was discovered in [36], which derived an $\mathcal{O}(\varepsilon^{-2/(2-\alpha)})$ sample complexity for neural TD with some $\alpha \in (0, 1]$. However, only the $\mathcal{O}(\varepsilon^{-2})$ complexity with $\alpha = 1$ can be guaranteed theoretically. Any complexity with $\alpha < 1$ requires additional assumptions on the sparsity of the semi-gradients, which is hard to justify for neural TD with Gaussian initialization and restricted update region. There have also been recent works [23], [24] that provide analysis for variants of TD for smooth function approximation. However, they can only guarantee the parameters converge to a stationary point and are difficult to implement in practical, large-scale tasks.

Finally, we review the related works on Newton and GN type policy evaluation methods. This type of method can date back to the residual gradient (RG) method [37] proposed by Baird in 1995, which was originally designed for linear approximation. For historical reasons, it is often termed Sutton's RG method [38]. This method was recently extended to nonlinear and neural network parameterization [39], [40]. However, [39] only numerically validates the local convergence behavior, and [40] only provides asymptotic convergence to global optima under linear function approximation. Notably, this line of research on RG methods is GN methods based on MSBE (1). Without adopting the target parameter in (2), they all suffer the double sampling issue and are difficult to implement in practical environments.

II. GNTD LEARNING

A. Preliminaries

We consider the infinite-horizon discounted Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathbb{P}, r, \gamma)$, with finite state space $\mathcal{S} = \{1, 2, \dots, |\mathcal{S}|\}$, finite action space $\mathcal{A} = \{1, 2, \dots, |\mathcal{A}|\}$, reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow [-R_{\max}, R_{\max}]$, transition probability $\mathbb{P}$, and a discount factor $\gamma \in (0, 1)$. Let policy π be a mapping that returns a probability distribution $\pi(\cdot|s)$ over the action space $\mathcal{A}$, for any state $s \in \mathcal{S}$. Then, the state-action value function (Q -function) under policy π is

$$Q^\pi(s, a) := \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \cdot r(s_t, a_t) \mid s_0 = s, a_0 = a \right] \quad \forall s, a.$$

For any mapping $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$, we denote T^π as the Bellman operator

$$T^\pi Q(s, a) := r(s, a) + \gamma P^\pi Q(s, a) \quad \forall s, a$$

where $P^\pi Q(s, a) = \mathbb{E}[Q(s', a') \mid s' \sim \mathbb{P}(\cdot|s, a), a' \sim \pi(\cdot|s')]$.

B. GNTD Method

For notational simplicity, let us view the parameterized Q -function $Q(\theta)$ as a column vector mapping in $\mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ by stacking the columns of this $|\mathcal{S}| \times |\mathcal{A}|$ matrix mapping. We still use the pair (s, a) to index the element of this vector, with $Q(s, a; \theta)$ denoting the $(s + (a - 1)|\mathcal{S}|)$ th element in the vectorized Q -function. Recall the nonlinear least square subproblem of FQI

$$\min_{\theta} \|Q(\theta) - T^\pi Q(\theta)\|_\mu^2.$$

Unlike FQI, which solves this problem with sufficiently many SGD steps, and unlike TD, which optimizes the loss with merely one SGD step, we would like to propose a method lying between FQI and TD methods. Denote $J_Q(\theta)$ as the Jacobian matrix of the parameterized Q -function $Q(\theta)$. Then, the GNTD method linearizes the $Q(\theta)$ in the FQI subproblem and updates the iterates by

$$\begin{cases} d_*^k = \operatorname{argmin}_d \|Q(\theta^k) + J_Q(\theta^k)d - T^\pi Q(\theta^k)\|_\mu^2 \\ \theta^{k+1} = \theta^k + \beta \cdot d_*^k. \end{cases} \quad (3)$$

The underlying intuition is clear. Suppose the range space of the Jacobian matrix $J_Q(\theta^k)$ is expressive enough that there exists d_*^k such that $T^\pi Q(\theta^k) - Q(\theta^k) = J_Q(\theta^k)d_*^k$. Given proper smoothness conditions, informally, one also has

$$\|Q(\theta^k + \beta d_*^k) - Q(\theta^k) - \beta J_Q(\theta^k)d_*^k\|_2 \leq \mathcal{O}(\beta^2)$$

where $\|\cdot\|_2$ is the ℓ_2 norm for vector and the spectral norm for matrix. Combining the above two inequalities yields

$$Q(\theta^{k+1}) = \beta \cdot T^\pi Q(\theta^k) + (1 - \beta) \cdot Q(\theta^k) + \mathcal{O}(\beta^2).$$

Then, the contraction of Bellman operator further yields

$$\begin{aligned} & \|Q(\theta^K) - Q^\pi\|_\infty \\ & \leq (1 - (1 - \gamma)\beta)^K \|Q(\theta^0) - Q^\pi\|_\infty + \mathcal{O}\left(\frac{\beta}{1 - \gamma}\right). \end{aligned}$$

Algorithm 1 GNTD Algorithm

Input: Distribution $\mathcal{D}$, Q function parameters θ^0 , batch size N , damping rate ω , learning rate β .

Initialize the Q network parameters θ^0 .

for $k = 0, 1, \dots, K - 1$ **do**

Obtain a dataset $\mathcal{D}_k = \{(s_i, a_i, r_i, s'_i, a'_i)\}_{i=1}^N$ of batch size N from data distribution $\mathcal{D}$.

Update θ^{k+1} by (7).

end for

Output: θ^K .

Therefore, setting $\beta = \mathcal{O}(\epsilon)$ or adopting a diminishing sequence of $\beta^k \rightarrow 0$ will provide the convergence of GNTD in this ideal situation.

For each tuple $\xi = (s, a, r, s', a') \sim \mathcal{D}$ where $\mathcal{D}$ is the distribution, $(s, a) \sim \mu$, $r = r(s, a)$, $s' \sim \mathbb{P}(\cdot|s, a)$, $a' \sim \pi(\cdot|s')$, we define the loss function

$$\ell(\xi, \theta^k, d) := \left(\nabla Q(s, a; \theta^k)^\top d + \delta^k(\xi) \right)^2 l.$$

Here $\delta^k(\xi) = Q(s, a; \theta^k) - (r(s, a) + \gamma Q(s', a'; \theta^k))$ denotes the TD error with respect to the tuple ξ . By removing terms irrelevant to d , the GNTD subproblem for d_*^k in (3) can be rewritten in a more sample-friendly form

$$d_*^k = \operatorname{argmin}_d L(\theta^k, d) := \mathbb{E}_{\xi \sim \mathcal{D}} [\ell(\xi, \theta^k, d)]. \quad (4)$$

Define the curvature matrix H and the semi-gradient g as

$$\begin{aligned} H(\theta^k) &:= \mathbb{E}_{(s, a) \sim \mu} \left[\nabla Q(s, a; \theta^k) \nabla Q(s, a; \theta^k)^\top \right] \\ g(\theta^k) &:= \mathbb{E}_{\xi \sim \mathcal{D}} [\delta^k(\xi) \cdot \nabla Q(s, a; \theta^k)]. \end{aligned} \quad (5)$$

Then, (4) has a closed form solution $d^k = -[H(\theta^k)]^{-1}g(\theta^k)$. Note that the solution d^k is the GN direction for solving the nonlinear system $Q(\theta) = T^\pi Q(\theta^k)$ [41], and the semi-gradient $g(\theta^k)$ is exactly the expected TD direction. Hence, we call our method the GNTD method.

Note that the population update (3) is not practically implementable. We introduce an empirical version of (4), with an additional quadratic damping term to improve robustness

$$d^k = \operatorname{argmin}_d \frac{1}{N} \sum_{\xi \in \mathcal{D}_k} \ell(\xi, \theta^k, d) + \frac{\omega}{2} \|d\|_2^2 \quad (6)$$

where $\mathcal{D}_k = \{(s_i, a_i, r_i, s'_i, a'_i)\}_{i=1}^N$ is a batch of N samples from $\mathcal{D}$. Then, (6) leads to the empirical GNTD update

$$\begin{cases} \hat{H}^k = \frac{1}{N} \sum_{\xi \in \mathcal{D}_k} \nabla Q(s, a; \theta^k) \nabla Q(s, a; \theta^k)^\top \\ \hat{g}^k = \frac{1}{N} \sum_{\xi \in \mathcal{D}_k} \delta^k(\xi) \cdot \nabla Q(s, a; \theta^k) \\ \theta^{k+1} = \theta^k - \beta \cdot (\hat{H}^k + \omega I)^{-1} \hat{g}^k \end{cases} \quad (7)$$

as detailed in Algorithm 1. For a large-scale function approximation class, such as over-parameterized neural networks, the matrix inversion in (7) can be expensive. In this case, we provide a practically efficient implementation of GNTD using the K-FAC method. Please see the details in Section V.

Before proceeding to Section III, let us provide a more detailed comparison of GNTD and the earlier GN approaches

under the research of RG methods, where one finds an update direction by

$$d_{\text{RG}}^k = \arg\min_d \|Q(\theta^k) - T^\pi Q(\theta^k) + \nabla(Q(\theta^k) - T^\pi Q(\theta^k))d\|_\mu^2.$$

This update corresponds to the GN direction at θ^k for solving the nonlinear system $Q(\theta) = T^\pi Q(\theta)$, and hence standard analysis applies. Unfortunately, the gradient vector of RG has the form of

$$g_{\text{RG}}(\theta^k) = \mathbb{E}_{\xi, \xi' \sim \mathcal{D}} [\delta^k(\xi) (\nabla Q(s, a; \theta^k) - \gamma \nabla Q(s'', a''; \theta^k))]$$

where $\xi = (s, a, r, s', a')$, $\xi' = (s, a, r, s'', a'')$ must share the same (s, a) pair. When constructing empirical updates like (7), this nature immediately leads to the difficulty of needing to sample two i.i.d. transitions from the same state action pair, which is not applicable in most of the practical environments.

By adopting an iteratively shifting target, GNTD avoids the need of double sampling issue suffered by RG type methods [37], [39], [40]. But this also leads to the semi-gradient nature of $\hat{g}^k$ in (7), causing standard GN analysis to fail. In Section III, we present how this difficulty can be resolved by adopting the inexact FQI perspective of our method.

III. CONVERGENCE RESULTS OF GNTD LEARNING

In this section, we provide a finite-time convergence analysis of the stochastic GNTD learning method under linear, neural network, and general smooth function approximations for Algorithm 1. The detailed proof can be found in Section IV in the Supplementary Material. First of all, let us make some basic assumptions about the data distribution.

Assumption 1: The data distribution μ is the stationary distribution over the state-action pairs under the policy π .

We write $U := \text{diag}(\mu)$ as an $|\mathcal{S}||\mathcal{A}|$ -dimensional diagonal matrix, whose (s, a) th diagonal element is $\mu(s, a)$. Denote $\langle f, g \rangle_\mu := \sum_{s,a} \mu(s, a) f(s, a) g(s, a)$, and $\|f\|_\mu^2 = \langle f, f \rangle_\mu$. Then, Assumption 1 indicates that T^π is a contraction with respect to $\|\cdot\|_\mu$ [19], that is,

$$\|T^\pi Q_1 - T^\pi Q_2\|_\mu \leq \gamma \|Q_1 - Q_2\|_\mu \quad \forall Q_1, Q_2 \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}| \times 1}.$$

For the ease of the notation, we regard Q^π as a $|\mathcal{S}||\mathcal{A}| \times 1$ column vector similar to $Q(\theta)$, satisfying the Bellman equation $Q^\pi = T^\pi Q^\pi$ in later discussion.

A. Linear Approximation

Consider the linear function approximation with $\theta \in \mathbb{R}^d$

$$Q(s, a; \theta) = \phi(s, a)^\top \theta \quad \forall s, a.$$

Let $\Phi \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}| \times d}$ be the collection of all feature vectors, whose (s, a) th row equals $\phi(s, a)^\top$. Without loss of generality, we assume $\|\phi(s, a)\|_2 \leq 1$ for any (s, a) . Then, the minimizer θ^* of the MSBE (1) satisfies the projected Bellman equation

$$\Phi \theta^* = \Pi_\mu T \Phi \theta^*$$

where Π_μ is the projection to the subspace $\{\Phi x : x \in \mathbb{R}^d\}$ under the inner product $\langle \cdot, \cdot \rangle_\mu$. We denote $Q^* := \Phi \theta^*$ as the optimal linear approximator.

In the under-parameterized regime where $d < |\mathcal{S}||\mathcal{A}|$, we make the following assumption for the feature covariance matrix, in accordance with [7].

Assumption 2: Let $\Sigma := \mathbb{E}_{(s,a) \sim \mu} [\phi(s, a) \phi(s, a)^\top]$ be the feature covariance matrix. We assume $\Sigma > 0$ and denote $\lambda_0 > 0$ as its minimum eigenvalue.

Then, the next theorem provides the convergence rate of stochastic linear GNTD in the under-parameterized regime, which matches the standard rate of linear TD under the same assumption [7].

Theorem 1: Suppose Assumptions 1 and 2 hold. For sufficiently small ε , if we set the damping rate $\omega = \mathcal{O}(\varepsilon^{0.5})$, the iteration $K = \mathcal{O}(\log(1/\varepsilon))$, the batch size $\Omega(\omega^{-2} \log(Kd/\delta)) = N = \tilde{\mathcal{O}}(\varepsilon^{-1})$, and the learning rate $\beta = (1 - \gamma)\lambda_0/4$, then for all iterations $\theta^k \in \mathbb{R}^d, k \leq K$, Algorithm 1 satisfies

$$\begin{aligned} \|Q(\theta^k) - Q^\pi\|_\mu &\leq \left(1 - \frac{(1 - \gamma)^2 \lambda_0}{8}\right)^k \|Q(\theta^0) - Q^*\|_\mu \\ &\quad + C_1 \left(\omega + \sqrt{\frac{\log(K/\delta)}{N}}\right) + \frac{\varepsilon_{\text{lin}}}{1 - \gamma} \end{aligned} \quad (8)$$

with probability at least $1 - \delta$ (w.p. $1 - \delta$), where $C_1 > 0$ is a constant and $\varepsilon_{\text{lin}} := \|\Pi_\mu Q^\pi - Q^\pi\|_\mu$ denotes the intrinsic error of linear model. In addition, the output $\theta^K \in \mathbb{R}^d$ satisfies $\|Q(\theta^K) - Q^\pi\|_\mu^2 \leq \mathcal{O}(\varepsilon) + \mathcal{O}(\varepsilon_{\text{lin}}^2)$.

If the true Q -function is realizable, namely,

$$Q^\pi = Q^* \in \text{Span}\{\Phi\} \quad (9)$$

then $\varepsilon_{\text{lin}} = 0$ in Theorem 1 and the estimated Q -functions $Q(\theta^k)$ converges linearly to the true Q -function Q^π . Consider the notion of the ε -optimal solution in [7], [10], and [11]: $\mathbb{E}[\|Q(\theta^K) - Q^\pi\|_\mu^2] \leq \varepsilon$ or $\|Q(\theta^K) - Q^\pi\|_\mu^2 \leq \varepsilon$ with high probability, and suppose the Q -function is realizable, then Theorem 1 can guarantee a sample complexity of $KN = \tilde{\mathcal{O}}(1/\varepsilon)$, which matches the best result of stochastic linear TD [7], [8].

Note that in Theorem 1, we adopt a constant batch size $N = \Omega(\omega^{-2} \log(Kd/\delta))$ to facilitate the use of union bound to control the failure probability δ . An alternative option to decouple N and K could be choosing a summable sequence of $\delta_k = 3\delta/(\pi^2(k+1)^2)$ s.t. $\sum_{k=0}^{+\infty} \delta_k = \delta/2$, and then selecting a time varying batch size $N_k = \Omega(\omega^{-2} \log(d/\delta_k))$ for any iteration $k \geq 0$.

B. Neural Network Approximation

For a two-layer neural network with width m , we write

$$Q(s, a; \theta) := \frac{1}{\sqrt{m}} \sum_{r=1}^m b_r \psi(\theta_r^\top \phi(s, a)) \quad (10)$$

where $\phi: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^d$ is the feature mapping, $\theta := (\theta_1^\top, \dots, \theta_m^\top)^\top \in \mathbb{R}^{m \times d}$, $b := (b_1, \dots, b_m)^\top \in \mathbb{R}^m$ are the weight matrices, and $\psi(z) := \max\{0, z\}$ is the ReLU activation. Similar to linear function approximation, we assume $\|\phi(s, a)\|_2 \leq 1$ for any (s, a) . We denote θ_r^k the value of θ_r in iteration k . For each r , we initialize the $\theta_r^0 \sim \mathcal{N}(0, \nu^2 I)$ for any given $\nu > 0$ and $b_r \sim \text{Unif}\{-1, +1\}$. The value of initialization parameter ν will affect the final choice of the network width parameter m , and the parameter b will not be trained during

the optimization. According to [33], we make the following assumption to ensure the positive definiteness of the Gram matrix.

Assumption 3: For all pairs $(s, a) \neq (s', a')$, we assume $\phi(s, a) \not\parallel \phi(s', a')$. Moreover, we assume $\mu(s, a) > 0$ to simplify the discussion.

The first setting in Assumption 3 implies independence and boundedness between two feature vectors. Regarding the latter setting, let $\Omega := \text{supp}(\mu)$ be the support of the distribution μ . Then, Assumption 3 is equivalent to requiring that the feature vectors are linearly independent and the support Ω covers the full $\mathcal{S} \times \mathcal{A}$. In general, $\Omega = \mathcal{S} \times \mathcal{A}$ is a very strong assumption. However, it is not necessary for our analysis. For a policy π , under mild conditions that the state transition Markov chain is aperiodic and irreducible, then $\mu(s, a) = 0$ iff. $\pi(a|s) = 0$. Then, the Bellman equation is *closed* on Ω since it does not involve $Q(s, a)$ for $(s, a) \notin \Omega$

$$Q^\pi(s, a) = r(s, a) + \sum_{(s', a') \in \Omega} \mathbb{P}(s'|s, a) \cdot \pi(a'|s') \cdot Q^\pi(s', a').$$

This allows us to only care about the MSBE on Ω

$$\min_{\theta} \mathbb{E} \left[\|Q(\theta) - T^\pi Q(\theta)\|_\mu^2 \right] = \mathbb{E} \left[\|Q(\theta) - T^\pi Q(\theta)\|_{\Omega, \mu}^2 \right]$$

where $\|f\|_{\Omega, \mu}^2 := \sum_{(s, a) \in \Omega} \mu(s, a) |f(s, a)|^2$. Therefore, Assumption 3 can actually be relaxed to requiring the μ -weighted Gram matrix on Ω to be positive definite. Though we will have no guarantee for $|Q(s, a; \theta) - Q^\pi(s, a)|$ for $(s, a) \notin \Omega$ in this case, it is not an issue. For example, when policy evaluation is applied as a built-in module of actor-critic methods, the Q -value outside Ω will never appear in the policy gradient (PG) formula. Therefore, we will assume $\Omega = \mathcal{S} \times \mathcal{A}$ for the ease of discussion.

Theorem 2: Suppose Assumptions 1 and 3 hold. If we set $\beta \in (0, 1)$, the damping rate $\omega = \mathcal{O}(\varepsilon^{0.5})$, the maximal iteration number $K = \mathcal{O}(\log(1/\varepsilon))$, the batch size $\Omega(\omega^{-2} \log(Km/\delta)) = N = \tilde{\mathcal{O}}(\varepsilon^{-1})$, and we set the width of the two-layer network (10) as $m = \Omega(\max\{|S|^3|A|^3\delta^{-2}, \varepsilon^{-3}\})$, then for all iterations $\theta^k \in \mathbb{R}^{m \times d}$, $k \leq K$, Algorithm 1 satisfies

$$\|Q(\theta^k) - Q^\pi\|_\mu \leq (1 - (1 - \gamma)\beta)^k \|Q(\theta^0) - Q^\pi\|_\mu + \frac{C_2}{(1 - \gamma)^2} \left(\frac{\log(NK/\delta)}{\sqrt{N}} + \omega + m^{-\frac{1}{6}} \right)$$

w.p. $1 - \delta$, where $C_2 > 0$ is a given constant. In addition, the output $\theta^K \in \mathbb{R}^{m \times d}$ satisfies $\|Q(\theta^K) - Q^\pi\|_\mu^2 \leq \mathcal{O}(\varepsilon)$.

Due to the strong representation power of neural networks, the intrinsic error of neural network parameterization admits an $\mathcal{O}(m^{-1/6})$ decay rate with respect to the network width. Then, Theorem 2 indicates an $KN = \tilde{\mathcal{O}}(\varepsilon^{-1})$ sample complexity for finding an ε -optimal solution. This result is better than the $\mathcal{O}(\varepsilon^{-2})$ sample complexity of neural TD [10], [11], and is also better than the $\mathcal{O}(\varepsilon^{-2/(2-a)})$, $a \in (0, 1]$ complexity of [36] while making no additional assumptions.

C. Smooth Function Approximation

In this subsection, we consider the smooth function approximation that satisfies the following properties.

Assumption 4: For all (s, a) , the function $Q(s, a; \cdot)$ is uniformly bounded by M , L_1 -Lipschitz, and L_2 -smooth

$$\begin{aligned} \|Q(s, a; \theta_1) - Q(s, a; \theta_2)\|_2 &\leq L_1 \|\theta_1 - \theta_2\|_2 \quad \forall \theta_1, \theta_2 \\ \|\nabla Q(s, a; \theta_1) - \nabla Q(s, a; \theta_2)\|_2 &\leq L_2 \|\theta_1 - \theta_2\|_2 \quad \forall \theta_1, \theta_2. \end{aligned}$$

Analogous to Assumption 3, we make the following requirement for the Jacobian matrix $J_Q(\theta)$.

Assumption 5: $\exists \lambda_0 > 0$ such that for any θ , we have $J_Q(\theta)^\top U J_Q(\theta) \geq \lambda_0 I$, where $U := \text{diag}(\mu)$ is diagonal.

When $Q(\cdot)$ is linear, Assumption 5 reduces to Assumption 3. It actually implies that the objective function $L(\theta^k, d)$ [see (4)] is $2\lambda_0$ -strongly convex with respect to the parameters d . Let us define the worst case optimal fitting error of $L(\theta, d)$ over the parameter space $\mathcal{F}$ of θ as

$$\varepsilon_{\mathcal{F}}^2 = \max_{\theta \in \mathcal{F}} \min_d L(\theta, d) < \infty \quad (11)$$

then we have the following theorem.

Theorem 3: Suppose Assumptions 1, 4, and 5 hold. Set $\beta, \omega = \mathcal{O}(\varepsilon^{0.5})$, the maximal iteration number $K = \tilde{\mathcal{O}}(\varepsilon^{-0.5})$, and the batch size $\Omega(L_1^4 \omega^{-2} \log(Kd/\delta)) = N = \tilde{\mathcal{O}}(\varepsilon^{-1})$, then w.p. $1 - \delta$, for all iterations $\theta^k \in \mathbb{R}^d$, $k \leq K$, Algorithm 1 satisfies

$$\begin{aligned} \|Q(\theta^k) - Q^\pi\|_\mu &\leq (1 - (1 - \gamma)\beta)^k \|Q(\theta^0) - Q^\pi\|_\mu \\ &\quad + \frac{C_3}{1 - \gamma} \left(\omega + \sqrt{\frac{\log(2K/\delta)}{N}} + \varepsilon_{\mathcal{F}} + \beta \right) \end{aligned}$$

for a given constant $C_3 > 0$. In addition, the output $\theta^K \in \mathbb{R}^d$ satisfies $\|Q(\theta^K) - Q^\pi\|_\mu^2 \leq \mathcal{O}(\varepsilon) + \mathcal{O}(\varepsilon_{\mathcal{F}}^2)$.

Theorem 3 suggests that stochastic GNTD can output an $(\varepsilon + \varepsilon_{\mathcal{F}}^2)$ -accurate Q -function estimate with $K = \tilde{\mathcal{O}}(\varepsilon^{-0.5})$ iterations and $KN = \tilde{\mathcal{O}}(\varepsilon^{-1.5})$ samples. To our best knowledge, there are no explicit finite-time convergence results of TD or FQI for general smooth functions in terms of finding ε -optimal solutions. Existing results, such as GTD2 and TDC [23], [24], only bound the complexity for finding ε -stationary points.

IV. EFFICIENT IMPLEMENTATION OF NEURAL GNTD

In order to facilitate an efficient implementation of the neural GNTD, we propose to incorporate in our method the K-FAC technique [28] that is designed for approximating matrix inversion in natural gradient and Newton-type methods in deep learning.

In Section II-B, we introduce the generalized GNTD algorithm, which encounters the issue of high complexity due to matrix inversion. We observe that GNTD has a similar structure to the natural gradient method, except that the natural gradient method, in terms of expression (natural gradient method requires the assumption that the loss function is the negative log-likelihood of a normal distribution). This observation inspires us to refine the neural GNTD algorithm.

For a feed-forward deep neural network (DNN) with L layers, we denote the weight matrices as θ_l of the l th layer ($l = 1, 2, \dots, L$) and we denote the ReLU activation function as $\psi(\cdot)$. For any state-action pair (s, a) , the output $Q(s, a; \theta)$ is in general a nonconvex function of the weights $\theta = [\theta_1^\top, \dots, \theta_L^\top]^\top$. Alternatively, θ_l can also be viewed as an $\mathbb{R}^{(n_{l-1}+1) \times n_l}$ parameter

Algorithm 2 Forward and Backward Pass of Q Function for a Single State-Action Pair (s, a)

```

1: Input:  $(s, a)$ , weights & biases  $\theta_l$  and ReLU function  $\psi$ .
2:  $p_0 = \phi(s, a)$ , where  $\phi$  is a feature mapping.
3: for  $l = 1, \dots, L$  do
4:    $\bar{p}_{l-1} = (p_{l-1}, 1)$ ;  $h_l = \text{Mat}(\theta_l)\bar{p}_{l-1}$ ;  $p_l = \psi(h_l)$ .
5: end for
6:  $\mathcal{D}p_L = e$ , where  $e$  is an all-ones vector.
7: for  $l = L, \dots, 1$  do
8:    $q_l = \mathcal{D}p_l \odot \psi'(h_l)$ ;  $-\text{DEL} -\text{DEL} \mathcal{D}\theta_l = \text{Vec}(q_l \bar{p}_{l-1}^\top)$ ;
    $\mathcal{D}p_{l-1} = \text{Mat}(\theta_l)^\top q_l$ .
9: end for

```

matrix that maps n_{l-1} -dimensional vectors to n_l -dimensional vectors. We define $\text{Mat}(\cdot)$ as a matrix form of the vector parameters related to the number of neurons in a single layer and define $\text{Vec}(\cdot)$ as a flattened vector form of the matrix parameters. Let $\odot$ and $\otimes$ be the Hadamard product and the Kronecker product, respectively. Then, following [28, Algorithm 1], the next algorithm describes the forward and backward pass of Q -value for a single state-action pair (s, a) .

From Algorithm 2, with the weights of the neural network being θ^k , we let p_l^k and q_l^k denote the forward vector and backward vector of the l th layer, respectively, and we define the matrices

$$P_l^k := p_l^k (p_l^k)^\top \quad \text{and} \quad Q_l^k := q_l^k (q_l^k)^\top. \quad (12)$$

For a training dataset that contains multiple data-points, the K-FAC method [28] attempts to approximate the matrix $H(\theta^k)$ in (5) by the following block-diagonal matrix:

$$H(\theta^k) \approx \text{diag}(\mathbb{E}[P_0^k] \otimes \mathbb{E}[Q_1^k], \dots, \mathbb{E}[P_{L-1}^k] \otimes \mathbb{E}[Q_L^k]).$$

After incorporating the identity matrix originated from the Levenberg–Marquardt method, we approximately calculate the matrix inversion as follows:

$$\begin{aligned}
& (H(\theta^k) + \omega I)^{-1} \\
& \approx \text{diag} \left((\mathbb{E}[P_0^k] \otimes \mathbb{E}[Q_1^k] + \omega I)^{-1} \right. \\
& \quad \left. \dots, (\mathbb{E}[P_{L-1}^k] \otimes \mathbb{E}[Q_L^k] + \omega I)^{-1} \right) \\
& \approx \text{diag} \left((\mathbb{E}[P_0^k] + \sqrt{\omega} I)^{-1} \otimes (\mathbb{E}[Q_1^k] + \sqrt{\omega} I)^{-1} \right. \\
& \quad \left. \dots, (\mathbb{E}[P_{L-1}^k] + \sqrt{\omega} I)^{-1} \otimes (\mathbb{E}[Q_L^k] + \sqrt{\omega} I)^{-1} \right).
\end{aligned}$$

For the stochastic sampling case where the expectations are approximated by the sample averages, we let $\hat{P}_l^{k-1/2}$ and $\hat{Q}_l^{k-1/2}$ be the empirical estimators of P_l^k and Q_l^k , given as

$$\hat{P}_l^{k-1/2} = \frac{1}{N} \sum_{i=1}^N P_l^k(i) \quad \text{and} \quad \hat{Q}_l^{k-1/2} = \frac{1}{N} \sum_{i=1}^N Q_l^k(i)$$

where $P_l^k(i)$ and $Q_l^k(i)$ are constructed by running Algorithm 2 for the i th data point and utilize (12). Then, we adopt a momentum scheme to construct $\hat{P}_l^k = (1 - \eta)\hat{P}_l^{k-1} + \eta\hat{P}_l^{k-1/2}$ and $\hat{Q}_l^k = (1 - \eta)\hat{Q}_l^{k-1} + \eta\hat{Q}_l^{k-1/2}$, which will provide better numerical stability than using direct sample average. Such a momentum update is also adopted in the implementation of

Algorithm 3 GNTD Algorithm With K-FAC (GNTD-KFAC)

Input: Distribution $\mathcal{D}$, Q function of neural network with parameters θ^0 , batch size N , damping rate ω , momentum parameter η , learning rate β .
Initialize the Q network parameters θ^0 .

for $k = 0, 1, \dots, K - 1$ **do**

Obtain a dataset $\mathcal{D}_k = \{(s_i, a_i, r_i, s'_i, a'_i)\}_{i=1}^N$ of batch size N from data distribution $\mathcal{D}$.

for $l = 1, 2, \dots, L$ **do**

if $k = 0$ **then**

$$\text{Compute estimates} \quad \begin{cases} \hat{P}_l^k = \frac{1}{N} \sum_{i=1}^N P_l^k(i), \\ \hat{Q}_l^k = \frac{1}{N} \sum_{i=1}^N Q_l^k(i). \end{cases}$$

else

$$\text{Compute estimates} \quad \begin{cases} \hat{P}_l^{k-1/2} = \frac{1}{N} \sum_{i=1}^N P_l^k(i), \\ \hat{Q}_l^{k-1/2} = \frac{1}{N} \sum_{i=1}^N Q_l^k(i). \end{cases}$$

$$\text{Update estimates} \quad \begin{cases} \hat{P}_l^k = (1 - \eta)\hat{P}_l^{k-1} + \eta\hat{P}_l^{k-1/2}, \\ \hat{Q}_l^k = (1 - \eta)\hat{Q}_l^{k-1} + \eta\hat{Q}_l^{k-1/2}. \end{cases}$$

end if

Update θ_l^{k+1} for each layer by (13) and (14).

end for

end for

Output: θ^K .

the standard K-FAC methods, see [28]. Similarly, let $\hat{g}^k$ [see (7)] be an estimator of the semi-gradient $g(\theta^k)$ [see (5)], and let $\hat{g}_l^k$ be the semi-gradient of the l th layer. Then, the descending direction for the l th layer is

$$\begin{aligned}
d_l^k & \approx (\hat{H}_l^k + \omega I)^{-1} \hat{g}_l^k \approx (\hat{P}_l^k + \sqrt{\omega} I)^{-1} \otimes (\hat{Q}_l^k + \sqrt{\omega} I)^{-1} \hat{g}_l^k \\
& = \text{Vec} \left((\hat{Q}_l^k + \sqrt{\omega} I)^{-1} \text{Mat}(\hat{g}_l^k) (\hat{P}_l^k + \sqrt{\omega} I)^{-1} \right). \quad (13)
\end{aligned}$$

Then, we naturally get the expression for the update

$$\theta_l^{k+1} = \theta_l^k - \beta d_l^k, \quad \text{for } l = 1, 2, \dots, L. \quad (14)$$

See Algorithm 3 for more details.

Finally, we provide a detailed comparison of per-iteration computational efficiency between GNTD with and without the K-FAC implementation. Take the two-layer NN, for example. In the trained layer, GNTD uses K-FAC for memory and computational efficiency. It approximates the Hessian matrix by a Kronecker product between two small matrices. Hence, the inverse of these matrices is much cheaper. In particular, a fully connected layer which maps an m -dimensional input vector to an n -dimensional output vector will generate a parameter matrix of the size mn , whose Hessian has a size of $mn \times mn$. Meanwhile, K-FAC approximates it by the Kronecker product of two matrices of the size $m \times m$ and $n \times n$, leading to a per-iteration computational complexity of $\mathcal{O}(m^3 + n^3)$. On the contrary, an exact computation of (7) without K-FAC approximation has a per-iteration computational complexity of $\mathcal{O}(m^3 n^3)$. Even if one applies an iterative algorithm such as

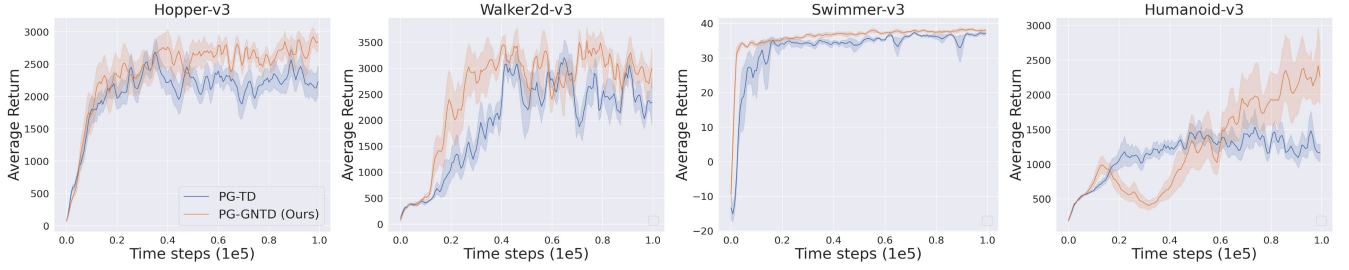


Fig. 1. Training curves over five random seeds on OpenAI Gym MuJoCo tasks. We use TD and GNTD to handle policy evaluation in the PG algorithm, respectively. The shaded area captures the standard deviation at each iteration. “Time steps” refers to the number of times that an algorithm samples a batch of N from the dataset. This quantity can be viewed as an indicator of the total sample consumption along the iterations.

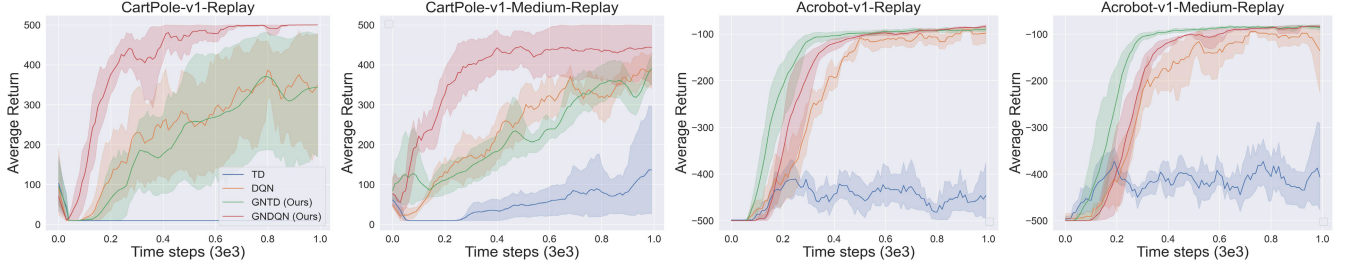


Fig. 2. Training curves over five random seeds on OpenAI Gym offline discrete tasks. The shaded area captures the standard deviation at each iteration.

TABLE I

UPPER PART OF THE TABLE SHOWS THE AVERAGE BELLMAN ERROR OF FOUR ALGORITHMS, TD, DQN, GNTD, AND GNDQN ON VARIOUS DATASETS, WHILE THE LOWER PART OF THE TABLE SHOWS THE AVERAGE TOTAL TIME (S) REQUIRED FOR THESE FOUR ALGORITHMS TO REACH THE CORRESPONDING REWARD THRESHOLD. HERE, CARTPOLE HAS A THRESHOLD OF -400 , WHILE ACROBOT HAS A THRESHOLD OF -100

Data Set	TD	DQN	GNTD	GNDQN
	<i>Bellman Error</i>			
CartPole-rep	757.93	0.6	3.03	0.51
CartPole-med-rep	97.78	0.66	1.51	0.58
Acrobot-rep	1.32	0.63	0.71	0.52
Acrobot-med-rep	1.41	0.68	0.68	0.58
	<i>Total Time (s)</i>			
CartPole-rep	—	23.17	26.59	15.46
CartPole-med-rep	—	36.65	30.73	26.96
Acrobot-rep	—	103.55	56.21	99.98
Acrobot-med-rep	—	95.67	46.33	89.71

conjugate gradient (CG) method to compute θ^{k+1} by solving linear system $(\hat{H}^k + \omega I)(\theta^{k+1} - \theta^k) = -\beta \hat{g}^k$, a single CG iteration still suffers a numerical cost of $\mathcal{O}(m^2 n^2)$, which is still worse than K-FAC. When dealing with a general L -layer neural network, the block-diagonal nature of K-FAC approximation allows the computational cost to scale linearly with respect to the depth L , while using the full Hessian matrix will result in an additional polynomial scaling in L .

Note that another commonly used technique in the practical implementation is the delayed update. That is, the coefficient matrices in K-FAC are updated and inverted only every 20~50 steps during each epoch. When the batch size $N \ll m, n$, one can further apply the SMW formula to compute $(\hat{Q}_i^k + \sqrt{\omega}I)^{-1}$ and $(\hat{P}_i^k + \sqrt{\omega}I)^{-1}$. With all these computational engineering, the per-iteration time and space complexity of our GNTD

method is very much comparable to that of the TD method on average, after adopting the K-FAC approximation technique. Nevertheless, due to the amount of engineering, there is still a gap between the theoretical results of Theorem 2 and the practical implementation of Algorithm 3. It remains an interesting future direction to study the theoretical guarantee of GNTD with K-FAC approximation. In this article, we will numerically evaluate the efficiency of this method.

V. EXPERIMENTS

Finally, we conduct a series of experiments over the OpenAI Gym [45] tasks and demonstrate the efficiency of the GNTD method under a variety of settings. For the sake of simplicity, we default to Algorithm 3 (i.e., GNTD-KFAC) as the implemented version of GNTD in this section.

In detail, we first examine the advantage of GNTD over TD in an on-policy RL setting, where the policy evaluation serves as a built-in module of the policy iteration method. Second, we also consider a few offline RL tasks, where we extend the proposed method to the Q -learning settings. All the compared learning algorithms are trained without exploration. We compare the performance of different algorithms in terms of the Bellman error and the final return. We use deterministic actions when evaluating the policies. Each algorithm undergoes 100 test runs per evaluation, and we report both mean rewards and standard deviations at convergence.

A. Policy Optimization With GNTD Method

First, we present the experiments where GNTD and TD are executed as built-in modules of an entropy regularized [18] policy iteration method. Typically, policy iteration is divided into two steps: policy evaluation and policy improvement. In detail, given an initial policy π_0 , our agents collect a data batch and then perform a 25-step policy evaluation to obtain Q^{π_0} , by

TABLE II

AVERAGE NORMALIZED SCORE OVER THE FINAL TEN EVALUATIONS AND FIVE SEEDS. UNDER THE SAME CONFIGURATION, WE RUN THE FOUR ALGORITHMS OF BC, BCQ, TD3 +BC, AND GNTD3 +BC, AND ALSO USE THE RESULTS REPORTED BY CQL AND TD3 +BC BEFORE. $\pm$ CAPTURES THE STANDARD DEVIATION. “m,” “m - r,” “m - e,” AND “e,” RESPECTIVELY, DENOTE “MEDIAN,” “MEDIAN-REPLAY,” “MEDIAN-EXPERT,” AND “EXPERT”

	BC	BCQ	CQL [46]	TD3+BC	GNTD3+BC(Ours)
Halfcheetah-m	42.4 $\pm$ 0.2	47.2 $\pm$ 0.4	37.2	46.5 $\pm$ 17.6	56.7$\pm$ 0.3
Hopper-m	30.1 $\pm$ 0.3	34.0 $\pm$ 3.8	44.2	100.2$\pm$ 0.2	100.5$\pm$ 0.3
Walker2d-m	12.6 $\pm$ 3.1	53.3 $\pm$ 9.1	57.5	79.4 $\pm$ 1.6	81.1$\pm$ 1.3
Halfcheetah-m-r	34.5 $\pm$ 0.8	33.0 $\pm$ 1.7	41.9	42.2$\pm$ 0.8	42.6$\pm$ 0.4
Hopper-m-r	20.0 $\pm$ 3.1	28.6 $\pm$ 1.1	28.6	31.8$\pm$ 2.0	32.4$\pm$ 1.3
Walker2d-m-r	8.1 $\pm$ 1.3	11.5 $\pm$ 1.3	15.8	23.9$\pm$ 1.6	24.8$\pm$ 2.1
Halfcheetah-m-e	70.6 $\pm$ 7.1	84.4 $\pm$ 4.5	27.1	90.9 $\pm$ 3.6	98.2$\pm$ 3.3
Hopper-m-e	92.5 $\pm$ 15.1	111.4 $\pm$ 1.2	111.4	111.9$\pm$ 0.4	112.0$\pm$ 0.1
Walker2d-m-e	12.2 $\pm$ 3.3	50.7 $\pm$ 7.2	68.1	94.6 $\pm$ 15.5	104.5$\pm$ 4.4
Halfcheetah-e	104.9 $\pm$ 1.4	96.6 $\pm$ 2.8	82.4	103.9 $\pm$ 1.7	107.6$\pm$ 0.6
Hopper-e	111.3 $\pm$ 0.9	108.7 $\pm$ 5.1	111.2	112.3$\pm$ 0.1	112.2$\pm$ 0.5
Walker2d-e	58.1 $\pm$ 9.1	92.6 $\pm$ 5.0	103.8	105.2 $\pm$ 1.8	107.6$\pm$ 1.5
Total	597.3 $\pm$ 37.7	752.0 $\pm$ 43.2	728.9	942.8 $\pm$ 46.9	980.2$\pm$ 16.1

either the GNTD or TD method. Then, we take a 1-step PG ascent to the entropy regularized total reward

$$\max_{\pi} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi(\cdot|s)} [Q^{\pi_0}(s, a) + \lambda H(\pi(\cdot|s))]$$

where $H(\pi(\cdot|s)) = \mathbb{E}_{a \sim \pi(\cdot|s)} [-\log \pi(a|s)]$ is the entropy of the density function $\pi(\cdot|s)$. Then, the agent executes the new policy to collect a new data batch, and loops through policy optimization until convergence.

We parameterize both the policy function π and state-action value function Q using two-layer neural networks, with the GNTD algorithm implemented using the K-FAC approximation for computational efficiency. The feature representation $\phi(s, a)$ has a dimensionality of 393 for the Humanoid-v3 environment in the online RL tasks, while ranging between 10 and 20 for other environments. During the model training, we employ a constant batch size of $N = 128$ and set the learning rate for the Q -function to $\beta = 0.0003$ for both TD and GNTD methods, with an additional damping rate of $\omega = 0.25$ specifically for GNTD. All the algorithms adopt a Gaussian initialization, and we terminate the algorithms after $1e^5$ iterations.

Fig. 1 shows the experimental results under several on-policy OpenAI gym environments. PG-GNTD and PG-TD refer to the policy optimization based on GNTD and TD, respectively. It can be observed that PG-GNTD converges faster than PG-TD in Hopper, Walker2d, and Swimmer environments. It also fetches higher final rewards in all tasks.

B. Offline RL Tasks

In this section, we present the experiments for both discrete and continuous offline RL tasks, where we will focus on optimizing rather than evaluating the policy. We compare the performance of our method against several benchmarks in terms of the Bellman error and average return.

1) *Discrete RL Tasks*: In this experiment, we present the experimental results under the OpenAI Gym CartPole-v1 and Acrobot-v1 environments. The tested algorithms include TD, DQN [1], GNTD, and GNDQN. In particular, both TD and GNTD are adapted to the Q -learning setting where the

Bellman operator is replaced with *optimal* Bellman operator. GNDQN is a variant of the GNTD method that incorporates a DQN-style momentum update to the target network, while taking GN steps with the K-FAC method to update the weight matrices. Namely, GNDQN adopts the same set of practical engineering techniques that adapt TD to DQN. All four algorithms share an identical neural network architecture, with a consistent learning rate of $\beta = 0.0003$ and a constant batch size of $N = 256$. For GNTD-based methods (GNTD and GNDQN), we set the damping rate $\omega = 0.25$. Both CartPole-v1 ($\phi(s, a)$, $\dim = 6$) and Acrobot-v1 ($\phi(s, a)$, $\dim = 9$) datasets contain 100 000 samples for standardized offline evaluation. All the algorithms adopt a Gaussian initialization, and we terminate the algorithms after $3e^3$ iterations.

Compared to the on-policy setting, offline RL requires strong conditions on the data distribution in order to obtain an optimal or near-optimal policy. According to [27] and [47], we consider the following types of datasets.

- 1) *Replay Datasets*: Train an online policy until convergence and use all samples during training.
- 2) *Medium-Replay Datasets*: Train an online suboptimal policy and use all samples during training.

From Fig. 2 and Table I, it can be observed that GNTD outperforms TD in terms of convergence speed, final reward, and Bellman error. After incorporating the momentum into the target network update, our GNDQN also dominates DQN in all the reported performance measures.

2) *Continuous Control Tasks*: Finally, we examine the performance of GNTD on two standard continuous control tasks from OpenAI Gym MuJoCo using D4RL datasets [48]. The D4RL dataset, built upon OpenAI Gym MuJoCo tasks, serves as a standard benchmark for offline RL. Each dataset typically features state dimensions of 10–20 and contains 100 K to 1 M samples. The dataset includes four variants as follows.

- 1) *Median (“-m”)*: Collected via medium-performance policies (between random and expert levels).

- 2) *Medium-Replay* (“ $-m-r$ ”): Experience replay buffer from medium-performance policies with exploration noise.
- 3) *Medium-Expert* (“ $-m-e$ ”): Hybrid data mixing medium- and expert-performance policies.
- 4) *Expert* (“ $-e$ ”): Generated exclusively by expert policies.

In these continuous control tasks, we propose a GNTD3+BC variant GNTD by implementing the same set of engineering techniques that adapt TD to TD3 +BC [49]. In detail, it adds a behavioral cloning regularization term to the expected total cumulative rewards

$$J_{\pi}(\phi) = \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[Q(s, \pi_{\phi}(s); \theta_1) - \lambda (\pi_{\phi}(s) - a)^2 \right].$$

For the critical part, we apply the GNTD method (K-FAC style) to minimize the following MSBE of the clipped double DQN [6], that is,

$$J_V(\theta_i) = \mathbb{E}_{\xi \sim \mathcal{D}} \left[(Q(s, a; \theta_i) - Q_{\text{targ}}(s, a))^2 \right], \quad i \in \{1, 2\}$$

where $Q_{\text{targ}}(s, a)$ is defined as

$$r(s, a) + \min \{ Q(s', \pi(s'); \theta_{1, \text{Ptarget}}), Q(s', \pi(s'); \theta_{2, \text{Ptarget}}) \}.$$

We update the relevant parameters by optimizing J_{π} and J_V .

For all the evaluated algorithms (BC, BCQ, TD3 +BC, and GNTD3 +BC), we maintain consistent training hyperparameters: batch size $N = 256$ and learning rate of $\beta = 0.0003$ for both actor and critic updates. The GNTD3 +BC implementation additionally employs a damping rate $\omega = 0.0003$ exclusively for critic updates. All the algorithms adopt a Gaussian initialization, and we terminate the algorithms after $1e^6$ iterations. Table II shows the final numerical results for GNTD3 +BC and other baselines, including BC (behavioral cloning), BCQ [50], CQL [46], and TD3 +BC [49]. Compared with TD3 +BC, GNTD3 +BC has higher final returns and lower variance in multiple environmental settings.

VI. CONCLUSION

In this article, we propose a GNTD learning method and study the finite-time analysis of our GNTD under i.i.d. sampling settings. We demonstrate that neural GNTD achieves an improved sample complexity of $\tilde{\mathcal{O}}(\varepsilon^{-1})$ compared to $\mathcal{O}(\varepsilon^{-2})$ of neural TD. An $\tilde{\mathcal{O}}(\varepsilon^{-1.5})$ sample complexity of GNTD is also established for general smooth function approximation. For future work, it is of interest to study the convergence properties of GNTD-type actor-critic learning algorithms. We also pay attention to the implementation efficiency of GNTD in practice, such as the approximation for matrix inversion.

REFERENCES

- [1] V. Mnih et al., “Playing Atari with deep reinforcement learning,” 2013, *arXiv:1312.5602*.
- [2] H. V. Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI Conf. Artif. Intell.*, 2016, vol. 30, no. 1, pp. 2094–2100.
- [3] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
- [4] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Proc. Adv. Neural Inf. Process. Syst.*, 1999, pp. 1008–1014.
- [5] T. P. Lillicrap et al., “Continuous control with deep reinforcement learning,” 2015, *arXiv:1509.02971*.
- [6] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
- [7] J. Bhandari, D. Russo, and R. Singal, “A finite time analysis of temporal difference learning with linear function approximation,” in *Proc. Conf. Learn. Theory*, 2018, pp. 1691–1692.
- [8] S. Zou, T. Xu, and Y. Liang, “Finite-sample analysis for SARSA with linear function approximation,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8665–8675.
- [9] R. Srikant and L. Ying, “Finite-time error bounds for linear stochastic approximation and TD learning,” in *Proc. Conf. Learn. Theory*, 2019, pp. 2803–2830.
- [10] Q. Cai, Z. Yang, J. D. Lee, and Z. Wang, “Neural temporal difference and q learning provably converge to global optima,” *Math. Oper. Res.*, vol. 49, no. 1, pp. 619–651, Feb. 2024.
- [11] P. Xu and Q. Gu, “A finite-time analysis of Q-learning with neural network function approximation,” in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 10555–10565.
- [12] A. Agazzi and J. Lu, “Temporal-difference learning with nonlinear function approximation: Lazy training and mean field regimes,” in *Proc. Math. Sci. Mach. Learn.*, 2019, pp. 37–74.
- [13] R. S. Sutton, “Learning to predict by the methods of temporal differences,” *Mach. Learn.*, vol. 3, no. 1, pp. 9–44, Aug. 1988.
- [14] S. J. Bradtko and A. G. Barto, “Linear least-squares algorithms for temporal difference learning,” *Mach. Learn.*, vol. 22, nos. 1–3, pp. 33–57, Jan. 1996.
- [15] R. S. Sutton et al., “Fast gradient-descent methods for temporal-difference learning with linear function approximation,” in *Proc. 26th Annu. Int. Conf. Mach. Learn.*, Jun. 2009, pp. 993–1000.
- [16] R. S. Sutton, C. Szepesvári, and H. R. Maei, “A convergent $o(n)$ algorithm for off-policy temporal-difference learning with linear function approximation,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2009, pp. 1609–1616.
- [17] S. Tu and B. Recht, “Least-squares temporal difference learning for the linear quadratic regulator,” in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 5005–5014.
- [18] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [19] J. N. Tsitsiklis and B. V. Roy, “Analysis of temporal-difference learning with function approximation,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 9, 1996, pp. 1075–1081.
- [20] S. Bhatnagar, D. Precup, D. Silver, R. S. Sutton, H. R. Maei, and C. Szepesvári, “Convergent temporal-difference learning with arbitrary smooth function approximation,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 22, 2009, pp. 1204–1212.
- [21] J. Achiam, E. Knight, and P. Abbeel, “Towards characterizing divergence in deep Q-learning,” 2019, *arXiv:1903.08894*.
- [22] D. Brandfonbrener and J. Bruna, “Geometric insights into the convergence of nonlinear TD learning,” 2019, *arXiv:1905.12185*.
- [23] Y. Wang, S. Zou, and Y. Zhou, “Non-asymptotic analysis for two time-scale TDC with general smooth function approximation,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 9747–9758.
- [24] T. Xu and Y. Liang, “Sample complexity bounds for two timescale value-based reinforcement learning algorithms,” in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 811–819.
- [25] M. Riedmiller, “Neural fitted Q iteration—first experiences with a data efficient neural reinforcement learning method,” in *Proc. Eur. Conf. Mach. Learn.*, 2005, pp. 317–328.
- [26] J. Chen and N. Jiang, “Information-theoretic considerations in batch reinforcement learning,” in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1042–1051.
- [27] Z. Yang, Y. Xie, and Z. Wang, “A theoretical analysis of deep Q-learning,” in *Proc. Learn. Dyn. Control*, 2020, pp. 486–489.
- [28] J. Martens and R. Grosse, “Optimizing neural networks with kronecker-factored approximate curvature,” in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2408–2417.
- [29] Y. Pan, A. White, and M. White, “Accelerated gradient temporal difference learning,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 31, no. 1, 2017.
- [30] S. Ghiassian, A. Patterson, S. Garg, D. Gupta, A. White, and M. White, “Gradient temporal-difference learning with regularized corrections,” in *Proc. Int. Conf. Mach. Learn.*, vol. 1, 2020, pp. 3524–3534.
- [31] J. A. Boyan, “Technical update: Least-squares temporal difference learning,” *Mach. Learn.*, vol. 49, nos. 2–3, pp. 233–246, Nov. 2002.
- [32] M. Ghavamzadeh, A. Lazaric, O. Maillard, and R. Munos, “Lstd with random projections,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 1, 2010, pp. 721–729.

- [33] S. S. Du, X. Zhai, B. Póczos, and A. Singh, "Gradient descent provably optimizes over-parameterized neural networks," 2018, *arXiv:1810.02054*.
- [34] G. Zhang, J. Martens, and R. Grosse, "Fast convergence of natural gradient descent for over-parameterized neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8080–8091.
- [35] Z. Allen-Zhu, Y. Li, and Z. Song, "A convergence theory for deep learning via over-parameterization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 242–252.
- [36] T. Sun, D. Li, and B. Wang, "Finite-time analysis of adaptive temporal difference learning with deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 19592–19604.
- [37] L. C. Baird, "Residual algorithms: Reinforcement learning with function approximation," in *Proc. 12th Int. Conf. Mach. Learn.*, 1995, pp. 30–37.
- [38] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, vol. 1. Cambridge, MA, USA: MIT Press, 1998.
- [39] M. Gottwald, S. Gronauer, H. Shen, and K. Diepold, "Analysis and optimisation of Bellman residual errors with neural function approximation," 2021, *arXiv:2106.08774*.
- [40] A. Sharifnassab and R. Sutton, "Toward efficient gradient-based value estimation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 30827–30849.
- [41] S. Wright and J. Nocedal, *Numerical Optimization*, vol. 35. Cham, Switzerland: Springer, 1999, p. 7.
- [42] G. Lan, *First-Order and Stochastic Optimization Methods for Machine Learning*. Cham, Switzerland: Springer, 2020.
- [43] J. A. Tropp, "An introduction to matrix concentration inequalities," *Found. Trends Mach. Learn.*, vol. 8, nos. 1–2, pp. 1–230, May 2015.
- [44] Y. Cao and Q. Gu, "Generalization error bounds of gradient descent for learning over-parameterized deep ReLU networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 4, Apr. 2020, pp. 3349–3356.
- [45] G. Brockman et al., "OpenAI gym," 2016, *arXiv:1606.01540*.
- [46] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-learning for offline reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1179–1191.
- [47] R. Agarwal, D. Schuurmans, and M. Norouzi, "An optimistic perspective on offline reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 104–114.
- [48] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4RL: Datasets for deep data-driven reinforcement learning," 2020, *arXiv:2004.07219*.
- [49] S. Fujimoto and S. Gu, "A minimalist approach to offline reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 20132–20145.
- [50] S. Fujimoto, "Off-policy deep reinforcement learning without exploration," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 2052–2062.



Zhifa Ke received the B.Sc. degree in information and computing science from Nanjing University, Nanjing, China, in 2021, and the M.Sc. degree in data science (mathematics) from Peking University, Beijing, China, in 2024.

He is currently an AI Algorithm Researcher with Ubiquant Investment (Beijing) Corporation Ltd., Beijing. His research interests include deep learning, reinforcement learning, and quantitative trading.



Junyu Zhang received the B.Sc. degree in computational mathematics from Peking University, Beijing, China, in 2015, and the Ph.D. degree in operations research from the University of Minnesota, Twin Cities, Minneapolis, MN, USA, in 2020.

He was a Post-Doctoral Research Fellow with Princeton University, Princeton, NJ, USA, in 2021. He is currently an Assistant Professor with the Department of Industrial Systems Engineering and Management, National University of Singapore, Singapore. His research interests include reinforcement

learning theory, stochastic optimization, saddle point problems, as well as the related complexity theory and information-theoretic lower bounds.

Dr. Zhang has won the APORS Young Researcher Best Paper Award in 2023. He has been serving as an Area Chair for ICLR since 2022.



Zaiwen Wen (Member, IEEE) received the Ph.D. degree in operations research from Columbia University, New York, NY, USA, in 2009.

He is currently a Changjiang Distinguished Professor with the Ministry of Education, Peking University, Beijing, China. He mainly studies optimization algorithms and theory, and their applications in machine learning. He was funded by the National Ten Thousand Talents Program for Science.

Dr. Wen was awarded China Youth Science and Technology Award in 2016 and Beijing Outstanding Youth Zhongguancun Award in 2020. He has been an Associate Editor of *Journal of Scientific Computing*, *Journal of the Operations Research Society of China*, and *CSIAM Transactions on Applied Mathematics*.